produces:

```
Line 0 is: This is line one
Line 1 is: This is line two
Line 2 is: This is line three
Line 3 is: And so on...
```

The cool thing about with_index is that if the receiving object properly defines each, then with_index can be chained to any iterator method, you can do map.with_index or find.with_index or whatever.

Let's look at one more useful iterator. The reduce method (which can also be referred to as inject for historical reasons), lets you accumulate a value across the members of a collection. It lets you reduce an array to a single scalar value. For example, you can sum all the elements in an array or find their product using code such as this:

```
[1,3,5,7].reduce(0) { |sum, element| sum + element }         # => 16
[1,3,5,7].reduce(1) { |product, element| product * element } # => 105
```

Here's how reduce works: the first time the associated block is called, the first argument to the block is set to the first argument passed to reduce and the second argument to the block is set to the first element in the collection. In this case, for sum, the first time through the block, sum is 0, the argument, and element is 1, from the collection. The block performs sum + element, returning 1.

The second and subsequent times the block is called, the first block argument is set to the value returned by the block on the previous call, while the second argument continues to be passed successive items from the collection. So, the next time through the block, sum is 1 and element is 3, and the block therefore returns 4. Next time, sum is 4 and the element is 5, returning 9, then next time, 9 and 7 return 16. The final value of reduce is the value returned by the block the last time it was called.

If reduce is called with no parameter, it uses the first element of the collection as the initial value and starts the iteration with the second value. This means that we could have written the previous examples like this:

```
[1,3,5,7].reduce { |sum, element| sum + element }         # => 16
[1,3,5,7].reduce { |product, element| product * element } # => 105
```

To make things shorter, instead of a block, you can pass it the name of the method you want to apply to successive elements of the collection. These examples work because, in Ruby, addition and multiplication are simply methods on numbers, and :+ is the symbol corresponding to the method +:

```
[1,3,5,7].reduce(:+) # => 16
[1,3,5,7].reduce(:*) # => 105
```

But for one of these examples, there's a shortcut:

```
[1,3,5,7].sum # => 16
```

(Array#product is also a method, but it does something different, it returns the cross-product of two arrays…)

```
[1,3,5,7].product([2, 4, 6]) # => [[1, 2], [1, 4], [1, 6], [3, 2], [3, 4], [3,
                             # .. 6], [5, 2], [5, 4], [5, 6], [7, 2], [7, 4], [7,
                             # .. 6]]
```